Feuille de TP n° 3

No-code & low-code – Développement assisté par LLM : les Tours de Hanoï

Avant de commencer

Développement assisté par LLM. Un grand modèle de langage (ChatGPT, Mistral, Gemini, Claude…) produit du code à partir d'une description en langage naturel : on décrit, il écrit, on teste. C'est le prolongement naturel du no-code, mais le code obtenu est seulement *plausible* : le développeur reste responsable de le lire, de le tester et de le faire corriger.

Objectifs du TP. Apprendre à formuler des requêtes efficaces (contexte, objectif, contraintes, format attendu), à itérer (tester, diagnostiquer, reformuler) et à produire ainsi une application web complète en HTML/CSS/JavaScript, le jeu des Tours de Hanoï, en mesurant ce que le LLM apporte, où il se trompe, et comment deux LLM diffèrent sur un même prompt.

Vocabulaire.

- **Prompt** : le message envoyé au LLM. Un bon prompt donne un *contexte* (ce qui existe déjà), un *objectif* précis, des *contraintes* (langage, fichiers, interdits) et un *format de réponse* (fichiers complets, fonctions modifiées seulement…).
- **Conversation** : le LLM voit les messages précédents de la conversation, et rien d'autre. Ce qu'il ne voit pas (votre fichier modifié à la main, l'erreur de la console), il faut le lui donner.
- **Itération** : envoyer, intégrer, tester, puis reformuler avec le symptôme observé. Le premier code généré est rarement le bon.
- **Hallucination** : une fonction, une propriété ou un comportement inventés, présentés avec assurance. Le navigateur, lui, ne se laisse pas convaincre : la console (touche F12) fait foi.

Mise en place

1. Choisir un LLM disposant d'une offre gratuite (ChatGPT, Mistral Le Chat, Gemini, Claude…) ; un second sera nécessaire à l'exercice 7.
2. Créer un dossier `hanoi` contenant trois fichiers vides : `index.html`, `style.css` et `script.js`. Le jeu s'ouvre en double-cliquant sur `index.html` : aucun serveur, aucune installation.
3. Ouvrir la console du navigateur (F12, onglet Console) et la garder ouverte pendant tout le TP : c'est là qu'apparaissent les erreurs JavaScript, à recopier telles quelles dans vos prompts.
4. Créer un fichier `prompts.md`, le journal du TP : pour chaque prompt, son numéro, l'exercice, le LLM utilisé, le texte du prompt, et le résultat (fonctionne, bug observé, nombre de reformulations nécessaires). Ce journal fait partie du rendu.

Règles du TP. Les prompts sont écrits par vous, en français ou en anglais. Jusqu'à l'exercice 7, on avance étape par étape : un besoin par prompt, pas de « fais-moi le jeu complet ». Le code généré est lu avant d'être intégré, et le jeu est testé dans le navigateur après chaque intégration. Contraintes techniques imposées au LLM dans chaque prompt : HTML, CSS et JavaScript purs (aucune bibliothèque, aucun framework), trois fichiers séparés, pas de modules ES (`import/export`, bloqués lorsque la page est ouverte depuis un fichier local), interface en français.

Le jeu des Tours de Hanoï

Trois tours ; n anneaux de tailles différentes, empilés sur la première tour du plus grand au plus petit. On déplace un anneau à la fois, toujours celui du sommet d'une tour, et on ne peut le poser que sur une tour vide ou sur un anneau plus grand. Le but est de reconstituer la pile sur la troisième tour. Le nombre minimal de coups est $2^n - 1$, et la solution s'obtient par l'algorithme récursif classique :

```
hanoi(n, depart, arrivee, intermediaire) :
    si n = 0 : ne rien faire
    hanoi(n - 1, depart, intermediaire, arrivee)
    déplacer l'anneau du sommet de depart vers arrivee
    hanoi(n - 1, intermediaire, arrivee, depart)
```

Écrire un bon prompt

Un prompt de développement contient, dans cet ordre : le **rôle** donné au LLM, le **contexte** (le projet, les fichiers existants, collés si nécessaire), l'**objectif** de l'étape, les **contraintes** techniques et le **format de réponse** attendu. Exemple pour l'exercice 1, à adapter :

```
Tu es un développeur web front-end. Je construis le jeu des Tours de Hanoï en HTML,
CSS et JavaScript purs (aucune bibliothèque, aucun framework, pas de modules ES),
en trois fichiers ouverts directement dans le navigateur : index.html, style.css
et script.js. Interface en français.
Étape 1 : crée la page minimale avec
- une zone d'identifiant "plateau" contenant trois tours (éléments de classe "tour"),
- un champ numérique d'identifiant "nb-anneaux" (min 1, max 8, valeur 3),
- un bouton d'identifiant "btn-demarrer" libellé "Démarrer".
Pas encore de logique de jeu. Donne le contenu complet des trois fichiers, sans
explication.
```

Pour corriger un bug, le prompt suivant décrit le *symptôme* (ce qui devait se passer, ce qui se passe), recopie le message de la console et, si la conversation ne contient plus la version courante du code, le colle à nouveau. Quand la conversation devient longue et confuse, on en ouvre une nouvelle avec le code complet à jour.

Exercice 1 Page web minimale

Objectif. Obtenir la page de base du jeu : trois tours, un bouton « Démarrer » et un champ pour choisir le nombre d'anneaux.

Résultat attendu

- `index.html` s'ouvre depuis le dossier et affiche trois tours côte à côte ;
- le champ numérique est limité à l'intervalle 1–8 et vaut 3 par défaut ;
- la console ne signale aucune erreur, y compris au clic sur le bouton.

Questions

A. Le LLM a-t-il respecté la séparation en trois fichiers, ou placé du CSS ou du JavaScript dans la page ? A-t-il ajouté des éléments non demandés ? Corriger par un second prompt si nécessaire.

B. Comparer les identifiants et classes générés à ceux demandés : pourquoi est-il utile de les imposer dès le premier prompt ?

Exercice 2 Génération dynamique des anneaux

Objectif. Au clic sur « Démarrer », créer les anneaux sur la première tour, correctement empilés, en nombre égal à la valeur du champ.

Imposer au LLM l'architecture suivante, dont tout le reste du TP dépend : l'état du jeu est une structure JavaScript (trois piles, par exemple un tableau de trois tableaux de tailles d'anneaux) et une fonction `rendre()` reconstruit l'affichage à partir de cet état. Le DOM n'est jamais la source de vérité.

Résultat attendu

- de 1 à 8 anneaux, de largeurs décroissantes du bas vers le haut ;
- un second clic sur « Démarrer » remet le jeu à zéro sans dupliquer les anneaux ;
- les tailles ne sont pas codées en dur dans le HTML.

Questions

A. Où le LLM a-t-il rangé l'état du jeu ? Si c'est dans le DOM (classes, attributs, ordre des éléments), demander la réécriture, et expliquer pourquoi cela compte pour les exercices suivants.

B. Que se passe-t-il pour une valeur hors bornes (0, 12, vide) ? Faire corriger.

Exercice 3 Déplacement des anneaux

Objectif. Jouer à la souris : un clic sélectionne l'anneau au sommet d'une tour, un second clic sur une tour l'y dépose si la règle le permet.

Résultat attendu

- seul l'anneau du sommet d'une tour est sélectionnable, et il est mis en évidence ;
- le dépôt sur un anneau plus petit est refusé, avec un retour visuel, sans modifier l'état ;
- le dépôt sur une tour vide est accepté ; un clic sur la tour d'origine annule la sélection ;
- la règle est isolée dans une fonction `deplacementValide(de, vers)` qui ne touche pas au DOM.

Questions

A. Tester `deplacementValide` directement dans la console sur des piles fabriquées à la main (tour vide, sommet plus petit, sommet plus grand). Pourquoi demander une fonction séparée plutôt qu'un test enfoui dans le gestionnaire de clic ?

B. Relever un cas que le LLM n'avait pas prévu (clic sur un anneau qui n'est pas au sommet, double clic, clic pendant qu'aucune partie n'est lancée) et le lui faire traiter.

Exercice 4 Améliorations visuelles

Objectif. Rendre le jeu agréable : mise en page, couleurs des anneaux, transitions, retour visuel de la sélection.

Résultat attendu

- chaque anneau a une couleur distincte et la sélection est clairement visible ;
- la page reste lisible sur une fenêtre étroite ;
- le comportement du jeu est strictement inchangé et la console reste vide.

Questions

A. Le LLM a-t-il modifié `script.js` alors que seul le style était demandé ? Quelle contrainte ajouter au prompt pour l'en empêcher ?

B. Demander deux propositions de style différentes dans un même prompt, puis en retenir une : en quoi le format de réponse demandé change-t-il la qualité du résultat ?

Exercice 5 Démo automatique

Objectif. Un bouton « Démo automatique » résout le jeu sous les yeux de l'utilisateur grâce à l'algorithme récursif.

Exiger deux fonctions distinctes : `resoudre(n, de, vers, via)` qui renvoie la liste des coups (des couples `[de, vers]`) sans rien afficher, et une fonction qui rejoue cette liste coup par coup avec un délai (`setTimeout`, ou `async`/`await` avec une fonction d'attente).

Résultat attendu

- la démo repart d'une partie neuve et se termine avec tous les anneaux sur la troisième tour ;
- 3 anneaux donnent 7 coups, 5 anneaux 31 coups ;
- les clics de l'utilisateur sont ignorés pendant la démo ;
- `resoudre(3, 0, 2, 1).length` vaut 7 dans la console.

Questions

A. Envoyer d'abord le prompt *sans* donner l'algorithme : le LLM le connaît-il ? Est-il correct ?

B. Pourquoi séparer le calcul de la liste des coups de leur exécution animée ? Que se passe-t-il si l'utilisateur clique sur « Démarrer » pendant la démo, et comment le LLM a-t-il traité ce cas ?

Exercice 6 Extensions obligatoires

Objectif. Compléter le jeu avec les six fonctionnalités ci-dessous, une par prompt, dans cet ordre, en testant entre chaque.

1. un compteur de coups, incrémenté en mode manuel comme en démo ;
2. la détection de la victoire et un message de félicitations ;
3. un score fondé sur le nombre minimal de coups $2^n - 1$ (par exemple le rapport entre le minimum et le nombre de coups joués) ;

4. un bouton « Reset » ;
5. des animations avancées : l'anneau se soulève, glisse et redescend, plutôt que de réapparaître ;
6. un mode sombre / clair, avec un bouton pour basculer.

Résultat attendu

- le compteur et le score sont exacts après une partie manuelle et après une démo ;
- la victoire n'est détectée qu'avec tous les anneaux sur la troisième tour, une seule fois ;
- « Reset » remet compteur, score, sélection et affichage à zéro, y compris pendant une démo ;
- le mode sombre s'applique à toute la page et survit à un « Reset ».

Questions

A. Quelle technique le LLM a-t-il choisie pour l'animation (transitions CSS, `transform`, API `element.animate()`) ? Lui demander de justifier son choix et de proposer l'alternative.

B. Quelle fonctionnalité a demandé le plus de reformulations ? Formuler en une phrase ce qui manquait au premier prompt.

C. Comment le LLM écrit-il $2^n - 1$ en JavaScript ? Vérifier dans la console pour $n = 8$.

Exercice 7 Recommencer à zéro : un prompt unique

Objectif. Écrire un seul prompt, complet et structuré, qui fait générer l'intégralité du jeu (exercices 1 à 6), puis le tester sur deux LLM différents.

Consignes

1. Rédiger le prompt selon le plan de la partie « Écrire un bon prompt » : rôle, contexte, liste numérotée des fonctionnalités, contraintes techniques, architecture imposée (état séparé de l'affichage, `resoudre` séparée de l'animation), format de réponse (trois fichiers complets). L'enregistrer dans `prompt-final.md`.
2. L'envoyer, dans une conversation neuve, à deux LLM différents. Pour chacun, ouvrir le résultat sans le modifier et parcourir la liste des résultats attendus des exercices 1 à 6.
3. Corriger par des prompts successifs jusqu'à ce que tout fonctionne, en comptant les reformulations nécessaires pour chaque LLM.
4. Dresser un tableau comparatif : fonctionnalités correctes du premier coup, nombre de reformulations, taille du code, respect des contraintes, lisibilité.

Questions

A. Quelles parties du prompt ont eu le plus d'effet ? Retirer une contrainte (par exemple l'architecture imposée) et relancer : que change-t-il ?

B. Le code obtenu en un prompt est-il meilleur ou moins bon que celui construit en six exercices ? Sur quels critères ?

Exercice 8 Documentation automatique (optionnel)

Objectif. Faire ajouter, en tête de chaque fonction de `script.js`, un commentaire de documentation au format JSDoc, puis en vérifier l'exactitude.

Le prompt doit spécifier le format attendu, par exemple :

```
/**
 * Calcule la liste des coups qui déplace n anneaux d'une tour à une autre.
 * @param {number} n    Nombre d'anneaux à déplacer.
 * @param {number} de   Tour de départ (0, 1 ou 2).
 * @param {number} vers Tour d'arrivée.
 * @param {number} via  Tour intermédiaire.
 * @returns {number[][]} Liste de couples [de, vers], dans l'ordre d'exécution.
 */
function resoudre(n, de, vers, via) { ... }
```

Questions

A. Relire chaque docstring en regard du code : en trouver au moins une inexacte, vague ou incomplète (paramètre oublié, valeur de retour fausse, effet de bord non mentionné) et la corriger, à la main ou par un prompt ciblé.

B. Demander ensuite au LLM une page `tests.html` qui charge `script.js` puis un fichier `tests.js` vérifiant dans la console `resoudre` (nombre de coups, validité de chaque coup, position finale) et `deplacementValide`. Un test échoue-t-il ? Qui a tort, le test ou le code ?

Rendu. Une archive contenant le dossier `hanoi` (code complet, trois fichiers), une capture d'écran du jeu fonctionnel, le journal `prompts.md`, le `prompt-final.md` de l'exercice 7 avec le tableau comparatif des deux LLM, et une analyse d'une dizaine de lignes : ce que le LLM a apporté, où il s'est trompé, ce que vous avez dû comprendre vous-même, et ce que vous feriez autrement.